

Python 程式語言

Lecture 1: Introduction to Python and Development Environment Setup

Instructor: 顏家珩 Chia-Heng Yen

Course Roadmap

- Fundamentals & Environment
 - Introduction to Python and Development Environment Setup
 - Basic Syntax and Flow Control
 - Data Structures and Collections
- Advanced Programming
 - Modular and Functional Programming
 - File Handling and Exception Handling
 - Object-Oriented Programming
 - Algorithm Implementation in Python
 - Modules and Package Applications
- User Interface Development
 - GUI Programming with Tkinter and PyQt
- Data Processing & Analysis
 - Data Analysis Fundamentals
 - Scientific Computing Applications
 - Data Visualization
- AI & ML Applications
 - Introduction to Machine Learning with Applications
 - AI-Related Library in Python Applications

Outline

- Understand Python language features
- Set up complete development environment
- Write your first Python program
- Learn basic version control with Git

Python Overview

Creator of Python Programming Language

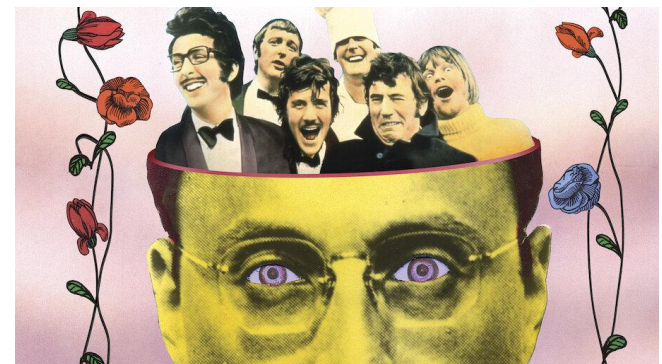
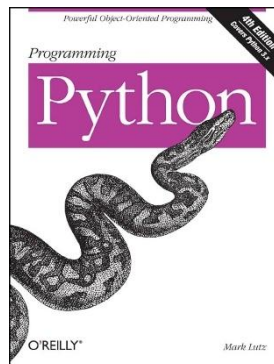
- Guido van Rossum (吉多·范·羅蘇姆)
 - Dutch programmer born in 1956, creator and “Benevolent Dictator for Life (仁慈的終身獨裁者)” (BDFL) of Python
 - Holds a Master's degree in Mathematics and Computer Science from University of Amsterdam (1982)
 - Work experience
 - Centrum Wiskunde & Informatica (CWI)
 - Python Creator
 - Google (2005-2012, code review systems)
 - Senior Software Engineer |
 - Dropbox (2013-2019)
 - Principal Engineer
 - briefly retired (2019-2020)
 - Microsoft (2020-present)
 - Distinguished Engineer



Ref: <https://computerhoy.20minutos.es/life/guido-van-rossum-creador-python-dice-milagro-complejo-sistema-funcione-1457251>

Python Creation Background

- Timeline: December 1989, during the Christmas week holiday period
- Initial Motivation: Van Rossum was looking for a "hobby" programming project to keep himself occupied while his office was closed for the holidays
- Design Philosophy: Envisioned as a descendant of the ABC programming language, but specifically designed to appeal to Unix/C hackers and developers
- Target Audience: Aimed to bridge the gap between ABC's simplicity and the practical needs of Unix/C programming community
- Naming Origin: Chose "Python" as the name in honor of the British comedy series "Monty Python's Flying Circus (蒙提·派森的飛行馬戲團)," not after the snake species



Major Python History Milestones

- December 1989: **Implementation began** - Guido van Rossum started developing Python at CWI during Christmas break
- February 1991: **First public release** - Python 0.9.0 published to alt.sources, included classes, inheritance, and exception handling
- January 1994: **Python 1.0 released** - Added functional programming tools (lambda, map, filter, reduce)
- October 2000: **Python 2.0 released** - Introduced list comprehensions, garbage collection, Unicode support, and transition to open-source development
- December 2001: Python 2.2 released - Unified types and classes into one hierarchy, added generators
- 2001: **Python Software Foundation (PSF) established** - Nonprofit organization formed to manage Python's development
- September 2006: Python 2.5 released - Introduced the **with** statement for context management

Major Python History Milestones

- December 2008: **Python 3.0 released** - Major backward-incompatible release to fix fundamental design flaws
- July 2018: Guido stepped down as BDFL - Transitioned to steering committee governance model
- January 2020: **Python 2 end-of-life** - Official support for Python 2.7 ended
- October 2021: Python 3.10 released - Introduced pattern matching feature
- October 2023: **Python 3.12 released** - Brought significant performance improvements

Why C/C++ Programmers Need Python?

- Rapid Prototyping and Development Speed
 - **Rapid Prototyping**: Quick algorithm testing and concept validation before C/C++ implementation
 - **Development Speed**: Minutes in Python vs hours in C++ for many tasks
 - Interactive Development: Python's REPL for immediate feedback and experimentation
- Complementary Strengths
 - **High-level Abstraction**: Automatic memory management and complex data structure handling, and system details automatically
 - **Rich Ecosystem**: Extensive libraries for I/O, networking, data processing, and more
 - Seamless Integration: Call C/C++ code using ctypes, Cython, or pybind11
- Industry Trends
 - DevOps Support: Excellent integration with modern CI/CD tools
 - **Data Analytics**: Optimize C++ applications through Python's analysis capabilities

Development Efficiency in Python

- Development Efficiency Comparison

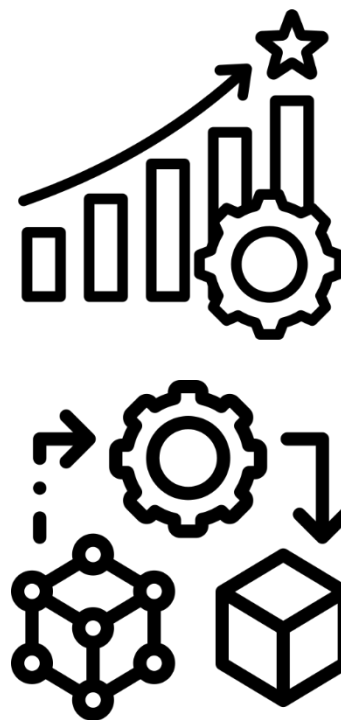
- C/C++ Strengths

- Performance
- Memory Control
- System Programming

- Python Advantages

- Rapid Prototyping
- Rich Ecosystem
- AI/ML Dominance
- Development Speed

- 1 line Python = 10 lines C++ !?



Development Efficiency Example

- Task: Read CSV and Calculate Average

- C++ Implementation

```
#include <iostream>
#include <fstream>
#include <vector>
#include <sstream>
#include <numeric>

int main() {
    std::ifstream file("data.csv");
    std::vector<double> values;
    std::string line, cell;
    while(std::getline(file, line)) {
        std::stringstream lineStream(line);
        while(std::getline(lineStream, cell, ',')) {
            values.push_back(std::stod(cell));
        }
    }
    double average = std::accumulate(values.begin(), values.end(), 0.0) /
values.size();
    std::cout << "Average: " << average << std::endl;
    return 0;
}
```

- Python Implementation

```
import pandas as pd

df = pd.read_csv('data.csv')
average = df['column'].mean()
print(f"Average: {average}")
```

Lines of Code

C++: ~20 lines

Python: 3 lines

Python vs Other Languages

Feature	Python	C++	Java	JavaScript
Learning Curve	Easy	Hard	Medium	Easy-Medium
Execution Speed	Slow	Very Fast	Fast	Medium-Fast
Development Speed	Very Fast	Slow	Medium	Fast
Memory Management	Auto	Manual	Auto	Auto
Primary Use Cases	AI/ML, Data Science, Backend, Automation	System Programming, Game Development, High-Performance Computing	Enterprise Applications, Android Development, Big Data	Web Development, Full-Stack, Mobile Apps, Desktop Apps

Trends in Programming Language Development

- Most Popular Programming Languages 1965 – 2022
 - <https://youtu.be/qQXXI5QFUfw?si=ZeLh3srniABgzkaA>

Top programming languages on GitHub

RANKED BY COUNT OF DISTINCT USERS CONTRIBUTING TO PROJECTS OF EACH LANGUAGE.

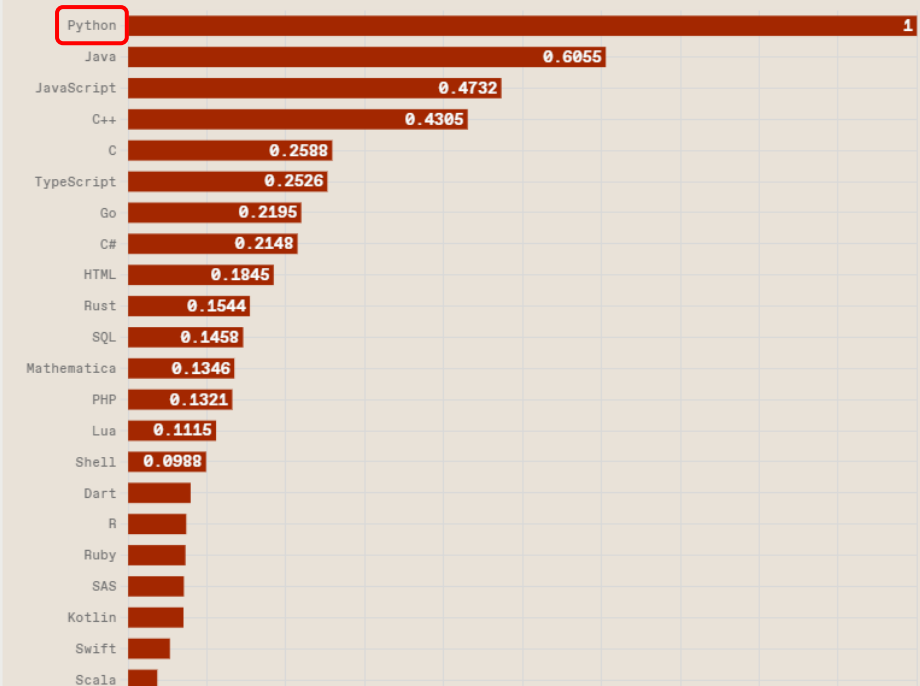


<https://www.bnext.com.tw/article/81172/python-2024-awesome>

Top Programming Languages 2024

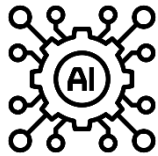
Click a button to see a differently weighted ranking

Spectrum **Trending** Jobs

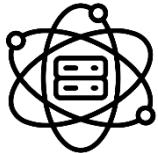


<https://spectrum.ieee.org/top-programming-languages-2024>

Python Application Domains



- Artificial Intelligence (Machine Learning & Deep Learning)
 - scikit-learn, PyTorch, TensorFlow



- Data Science
 - NumPy, pandas, matplotlib, seaborn, plotly



- Web Development (Backend Services)
 - Flask, Django, FastAPI



- Automation
 - Scripting, DevOps Tools, Testing Frameworks



- Scientific Computing
 - SciPy, SymPy



- Desktop Applications
 - tkinter, PyQt, Kivy



- Financial Technology, Network Programming, Game Development...

Industry Case Studies

- Instagram
 - Backend: Django (Python)
 - ML Recommendations
 - 2B+ Users
- Netflix
 - Content Recommendation
 - Data Analytics Pipeline
 - Python + Spark
- Tesla
 - Autopilot Data Processing
 - Python: Data Analysis
 - C++: Real-time Control
- Spotify
 - Music Recommendation
 - Backend Services (with JAVA)
 - Data Science Platform



TESLA



Python Ecosystem

- Python Package Index (PyPI)
 - 400,000+ Packages
- Popular Categories
 - Data & Analytics: pandas, NumPy, matplotlib, seaborn
 - Machine Learning: scikit-learn, TensorFlow, PyTorch
 - Web Frameworks: Django, Flask, FastAPI
 - Development Tools: pytest, black, mypy
 - Scientific Computing: SciPy, SymPy, Jupyter



Anaconda Installation & Configuration

Why Choose Anaconda?

- Anaconda Benefits
 - Package Management Simplified
 - Automatic dependency resolution
 - Conda-forge community support
 - Scientific Computing Ready
 - 500+ Pre-installed packages
 - NumPy, pandas, matplotlib included
 - Machine Learning libraries
 - Environment Isolation
 - Project-specific environments
 - Version conflict prevention
 - Easy environment switching
 - Cross-Platform Support
 - Performance Optimization
 - Beginner-Friendly
 - Professional Tools (IDE) Included



Anaconda vs Miniconda vs Python.org

- Distribution Comparison

Feature	Anaconda	Miniconda	Python.org
Download Size	~700MB	~50MB	~30MB
Installation Size	~3-5GB	~400MB	~100MB
Pre-installed Packages	300+	Essential	None
GUI Tools	Navigator	✗	✗
Package Manager	conda + pip	conda + pip	pip only
Best For	Beginners	Advanced	Minimalists

- Recommendation for Course
 - Anaconda - Complete solution for learning
 - All-in-one package
 - No setup complexity
 - Scientific computing ready
 - Beginner-friendly GUI

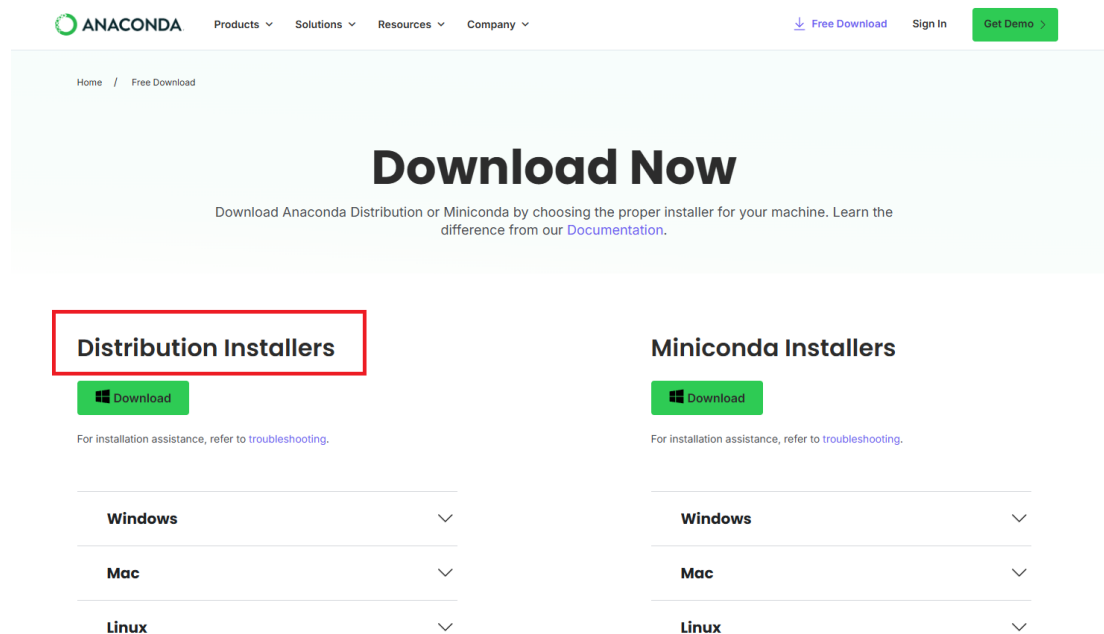
Windows Installation Steps

1. Download
2. Installation Options

Installation Steps

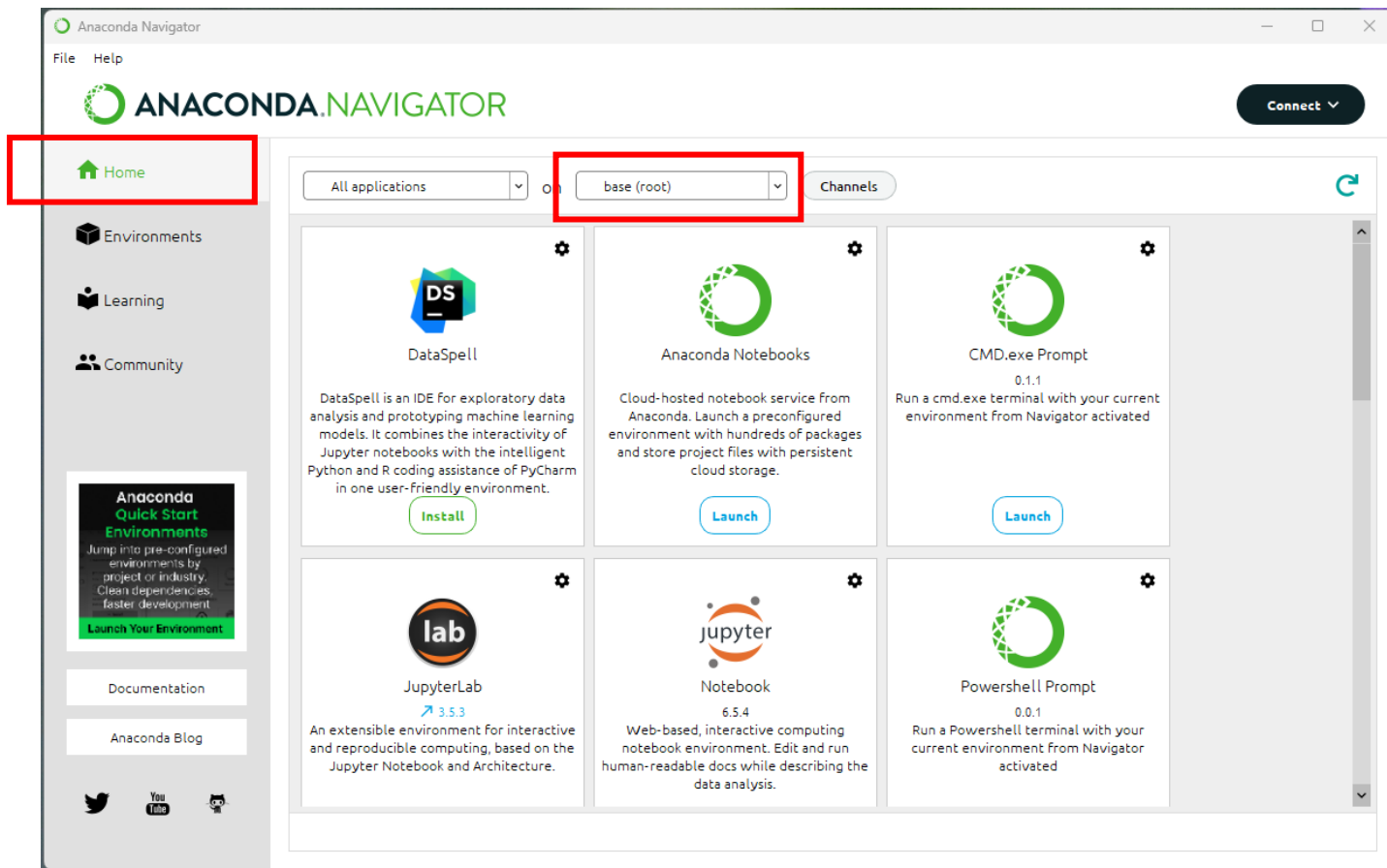
1. Download

- Visit: <https://www.anaconda.com/download/success>
- Choose Distribution Installers and Platform (Windows/Mac/Linux) Type



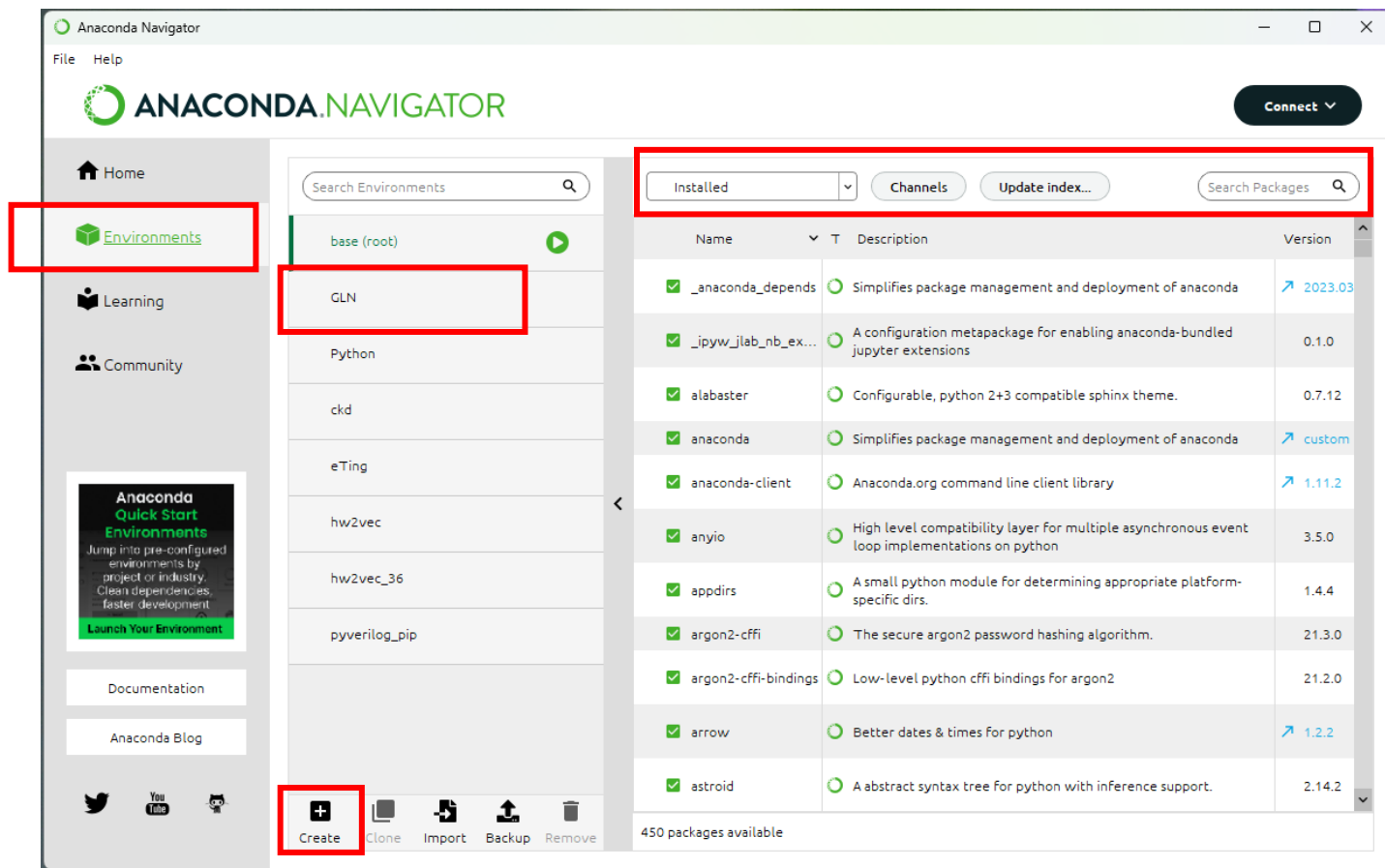
Anaconda Navigator Introduction

- Home Tab
 - Select environments & Launch applications



Anaconda Navigator Introduction

- Environments Tab
 - Manage conda environments & Install/Update/Remove packages

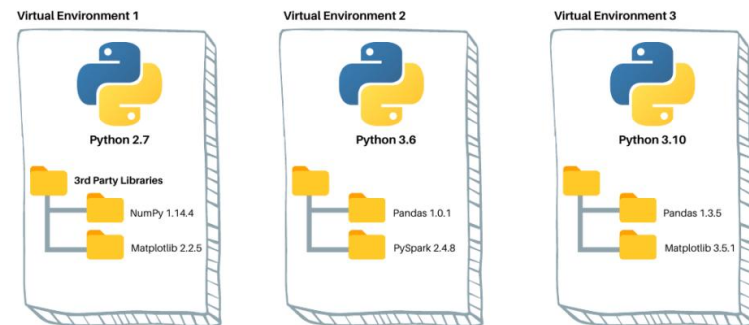


Basic Conda Commands

- Information Commands
 - `conda -version` # Check conda version
 - `conda info` # System information
 - `conda list` # List installed packages
 - `conda env list` # List environments
- Update Commands
 - `conda update conda` # Update conda itself
 - `conda update --all` # Update all
 - `conda update numpy` # Update specific package, e.g. numpy
- Search Commands
 - `conda search python` # Search for package
 - `conda search "*torch*"` # Wildcard search
- Configuration
 - `conda config --show` # Show configuration
 - `conda config --add channels conda-forge` # Add channel

Virtual Environment Concepts

- Why Virtual Environments?
- Problem Scenarios
 - Project A needs Django 3.2
 - Project B needs Django 4.1
- Solution: Environment Isolation
 - Base Environment: System Python installation, Shared packages
 - Project Environments: Isolated package versions, Project-specific dependencies
- Benefits
 - No version conflicts
 - Clean project setup
 - Easy deployment
 - Experiment safely



<https://www.dataquest.io/blog/a-complete-guide-to-python-virtual-environments/>

Environment Management

- Create New Environment
 - # Basic environment
`conda create --name myproject python=3.11`
 - # With specific packages
`conda create --name datascience python=3.11 pandas numpy matplotlib`
 - # From requirements file
`conda create --name myproject --file requirements.txt`
- Activate/Deactivate
 - # Activate environment
`conda activate myproject`
 - # Deactivate environment
`conda deactivate`
 - # Check current environment
`conda info --envs`
- Remove Environment
 - # Remove environment
`conda env remove --name myproject`
- Naming Convention
 - Use descriptive names
 - project-specific: web-app, ml-project
 - version-specific: python311, django4

Best Practices for Environment Management

- Environment Documentation
 - # Export environment
\$ conda env export > environment.yml
 - # Create from file
\$ conda env create -f environment.yml
 - # Export packages only
\$ conda list --export > requirements.txt
- Project Structure
 - my-project/
 - src/ # Source code
 - tests/ # Test files
 - environment.yml # Conda environment
 - requirements.txt # Pip packages
 - README.md # Documentation
- Performance Tips
 - Use mamba for faster solving
\$ conda install mamba -c conda-forge
\$ mamba install package_name
 - Regular cleanup
\$ conda clean --all

Package Manager Comparison (conda vs pip)

Feature	conda	pip
Package Source	Anaconda repos + conda-forge	PyPI
Binary Packages	Pre-compiled	Wheels available Some source builds
Dependency Resolution	Advanced (SAT solver)	Basic (improved in recent versions)
Non-Python Libraries	Includes (C/C++, R, etc.)	Python only
Virtual Environments	Built-in	Requires venv/virtualenv
Speed	Slower solving	Faster install
Package Count	~50,000 (conda-forge)	~500,000+

- When to Use
 - Use **conda** for Scientific computing, Complex dependencies, Environment management
 - Use **pip** for Latest package versions, Lightweight projects
- Best Practice: Hybrid Approach (**conda install major packages, pip install the rest**)

IDE Selection & Configuration

IDE Selection Guide

- For Different Users
 - Beginners
 - VS Code: User-friendly, popular, free
 - Spyder: MATLAB-like interface, scientific focus
 - Professional Developers
 - PyCharm: Full-featured, powerful debugging
 - VS Code: Lightweight, extensible, universal
 - Data Scientists
 - Jupyter Notebook: Interactive analysis, visualization
 - Spyder: Variable explorer, scientific libraries
 - System Administrators
 - vim/nano: Terminal-based, lightweight
 - VS Code: Remote development, SSH support

IDE Selection Guide

- Feature Comparison

Feature	VS Code	PyCharm	Spyder	Jupyter
Learning Curve	Easy	Medium	Easy	Easy
Memory Usage	Low	High	Medium	Low
Startup Time	Fast	Slow	Medium	Fast
Debugging	Good	Excellent	Good	Basic
Variable Explorer	Plugin	Built-in	Excellent	Limited
Git Integration	Excellent	Excellent	Basic	External
Extensions System	Excellent	Good	Limited	Moderate
Scientific Libraries	Good (with extensions)	Excellent	Excellent	Excellent
Team Collaboration	Excellent	Excellent	Basic	Good

- Recommendations
 - VS Code - Free, versatile, huge community
 - Spyder - Variable explorer

Git Version Control

Importance of Version Control

- Why Version Control?
- Without Version Control
 - my_project.py
 - my_project_v2.py
 - my_project_final.py
 - my_project_final_final.py
 - my_project_final_ACTUALLY_FINAL.py
 - my_project_this_is_really_the_final_version.py
- With Version Control
 - my_project.py (with complete history)

Key Benefits of Version Control

- Track Changes
 - Complete history of modifications (Who changed what and when)
 - Compare different versions
- Collaboration
 - Multiple developers on same project
 - Merge changes safely (Resolve conflicts systematically)
- Recovery
 - Recover deleted files
 - Restore previous working versions (Undo changes to any point in time)
- Branching
 - Experiment with new features safely
 - Parallel development streams
 - Easy switching between versions
- Industry Statistics
 - 95%+ of software projects use version control
 - Git is used by 87% of developers
 - GitHub hosts 100M+ repositories

Git vs Other Version Control

Feature	Git	SVN	Mercurial
Architecture	Distributed	Centralized	Distributed
Speed	Fast	Slow	Fast
Offline Work	Full	Limited	Full
Branching	Excellent	Poor	Good
Learning Curve	Steep	Easy	Medium
Market Share 市佔率	95%+	~3%	~2%
Popular Hosts 熱門託管	GitHub, GitLab, Bitbucket	SourceForge, Legacy systems	Perforce TeamHub

- Git Advantages for Students
 - Industry-standard skill
 - Free and open source
 - Excellent documentation
 - Large community support
 - Integration with all major IDEs

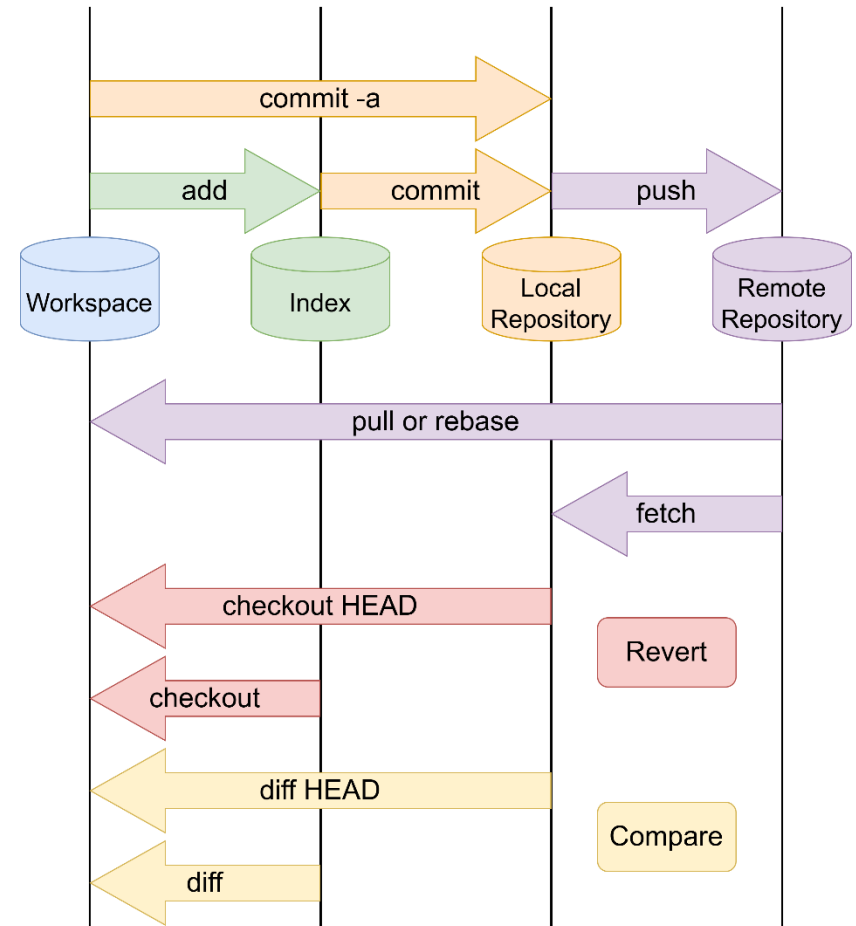


Git Basic Concepts

- Repository (Repo)
 - Container for your project
 - Stores complete history
 - Can be local or remote
- Commit
 - Snapshot of your project at a point in time
 - Includes author, timestamp, message
 - Unique identifier (SHA hash)
- Branch
 - Independent line of development
 - Default branch: main (formerly master)
 - Create branches for features/experiments
- Remote
 - Repository hosted on server
 - Common remotes: GitHub, GitLab, Bitbucket
 - Origin: default name for primary remote

Git Workflow

- Working Directory (Workspace)
 - Your actual project folder
 - Files as they exist on your computer
 - Where you make changes
 - File States
 - Untracked: New files Git doesn't know about
 - Modified: Changed files since last commit
 - Deleted: Files removed from working directory
- Staging Area (Index)
 - A preparation area for your next commit
 - Lives in `.git/index` file
 - Git's "shopping cart" for changes
- Local Repository
 - Complete project history on your machine (Contains all commits, branches, tags)
 - Lives in `.git` folder
- Remote Repository
 - Shared repository on server (GitHub, GitLab)
 - Central point for collaboration
 - Backup and synchronization hub

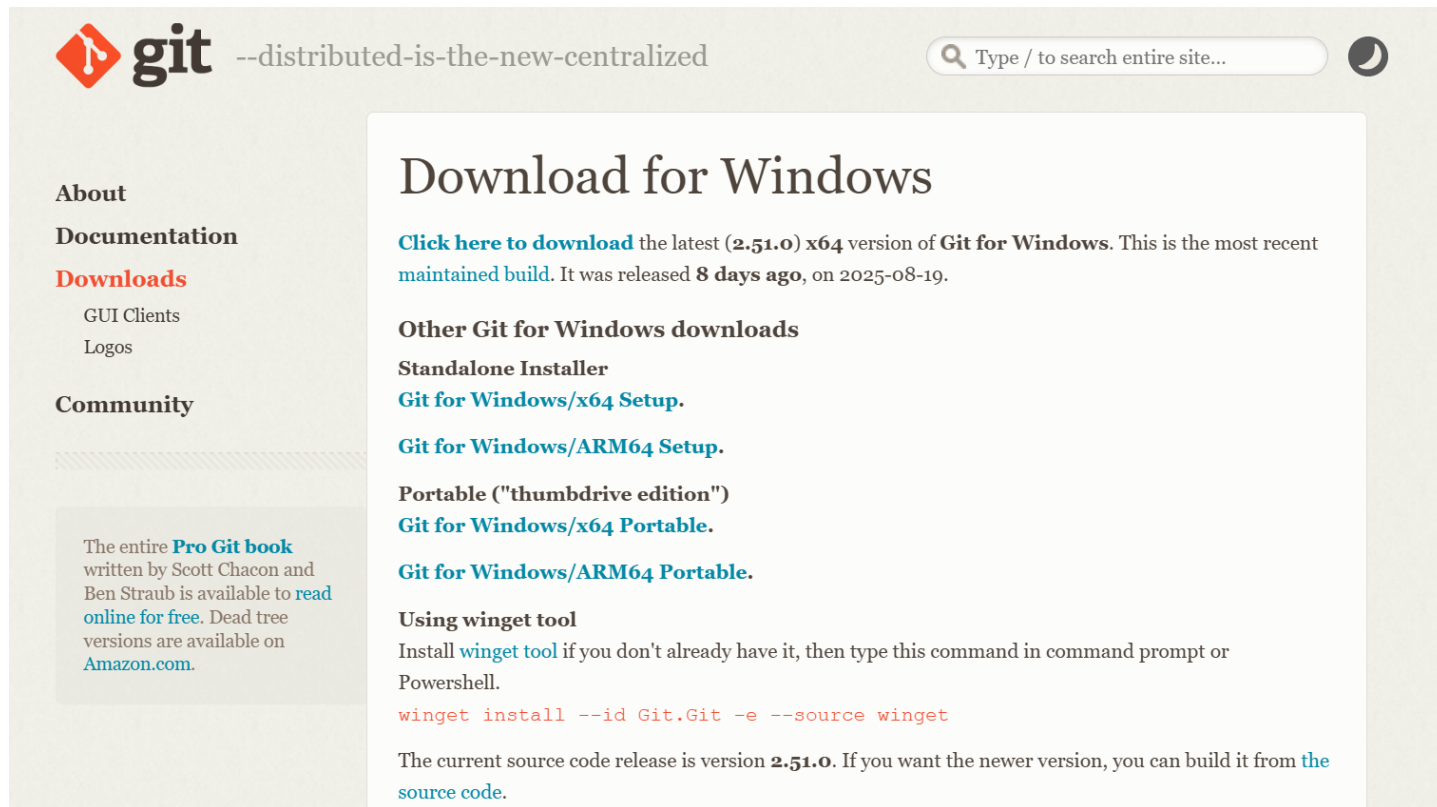


Git Installation

- Windows Installation
 1. Download Git for Windows: <https://git-scm.com/download/win>
 2. Installation Options: Use Git from command line and 3rd-party software, Use bundled OpenSSH, Use native Windows Secure Channel library
 3. Verification: `git --version`
- macOS Installation
 - Method 1: Download Git for macOS (Official Installer): <https://git-scm.com/download/mac>
 - Method 2: Homebrew (Recommended)
`/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`
`brew install git`
- Linux Installation
 - Debian/Ubuntu: `sudo apt update & sudo apt install git`
 - Red Hat/CentOS/Fedora (or for newer versions): `sudo yum install git` (`sudo dnf install git`)
- Verification Commands
 - `$ git --version` # Check Git version
 - `$ git config --list` # Show configuration
 - `$ which git` # Show Git location

Git Installation

- Windows Installation



The screenshot shows the Git website's 'Download for Windows' page. The header features the Git logo, the tagline '--distributed-is-the-new-centralized', a search bar, and a moon icon. The left sidebar contains links for 'About', 'Documentation', 'Downloads' (highlighted), 'GUI Clients', 'Logos', and 'Community'. A box in the sidebar promotes the 'Pro Git book'. The main content area is titled 'Download for Windows' and provides instructions for downloading the latest version (2.51.0) for Windows. It lists various download options: Standalone Installer, Git for Windows/x64 Setup, Git for Windows/ARM64 Setup, Portable ('thumbdrive edition'), and Git for Windows/x64 Portable. It also includes a section for using the winget tool with a command snippet and a note about the current source code release.

git --distributed-is-the-new-centralized

Type / to search entire site...

About

Documentation

Downloads

GUI Clients

Logos

Community

The entire **Pro Git book** written by Scott Chacon and Ben Straub is available to [read online for free](#). Dead tree versions are available on [Amazon.com](#).

Download for Windows

[Click here to download](#) the latest (**2.51.0**) **x64** version of **Git for Windows**. This is the most recent [maintained build](#). It was released **8 days ago**, on 2025-08-19.

Other Git for Windows downloads

Standalone Installer

[Git for Windows/x64 Setup.](#)

[Git for Windows/ARM64 Setup.](#)

Portable ("thumbdrive edition")

[Git for Windows/x64 Portable.](#)

[Git for Windows/ARM64 Portable.](#)

Using winget tool

Install [winget tool](#) if you don't already have it, then type this command in command prompt or Powershell.

```
winget install --id Git.Git -e --source winget
```

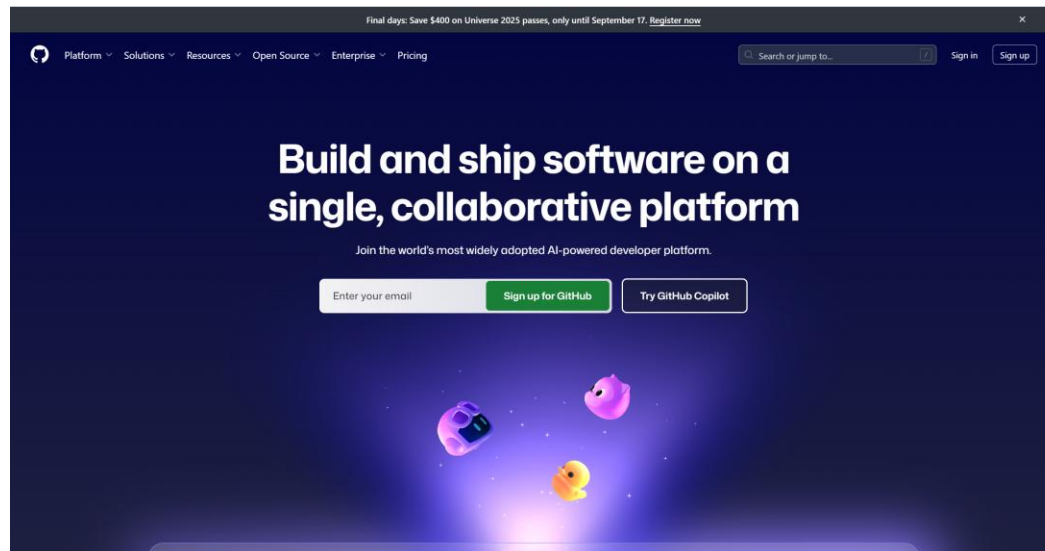
The current source code release is version **2.51.0**. If you want the newer version, you can build it from [the source code](#).

Git Initial Configuration

- Git Configuration Setup
 - User Identity
 - `git config --global user.name "Your Full Name"`
 - Set your email
 - `git config --global user.email "your.email@example.com"`

GitHub Account Creation

- What is GitHub?
 - Web-based Git repository hosting
 - Social coding platform
 - Free for public repositories
 - Used by 100M+ developers worldwide
- Visit: <https://github.com> & Account Registration



GitHub Account Creation

- SSH Key Setup
 - Why SSH?
 - Secure authentication
 - No password typing
 - Industry standard
 - 1. Generate SSH Key
 - Check for existing keys: `$ ls -la ~/.ssh`
 - Generate new key: `$ ssh-keygen -t ed25519 -C "your_email@example.com"`
 - Start SSH agent: `$ eval "$(ssh-agent -s)"`
 - Add key to agent: `$ ssh-add ~/.ssh/id_ed25519`
 - Copy public key: `$ cat ~/.ssh/id_ed25519.pub`
 - 2. Add to GitHub
 - 1. GitHub → Settings → SSH and GPG keys
 - 2. Click "New SSH key"
 - 3. Paste public key
 - 4. Test connection

First Git Repository

- Creating Your First Repository

- Local Repository Creation

- Create project directory

- \$ mkdir my-first-project

- \$ cd my-first-project

- Initialize Git repository

- \$ git init

- Check status

- \$ git status

- Project Structure

- my-first-project/

- ├── .git/ # Git metadata (hidden)
 - ├── README.md # Project description
 - ├── .gitignore # Files to ignore
 - └── src/ # Source code
 - └── main.py

First Git Repository

- First Commit Process
 - Add files to staging area
`$ git add README.md`
`$ git add src/main.py`
 - Or add all files
`$ git add .`
 - Check what's staged
`$ git status`
 - Create first commit
`$ git commit -m "Initial commit: Add README and main.py"`
 - View commit history
`$ git log`

Basic Git Workflow

- The Git Workflow
 - Daily Git Workflow
 1. Check Status
`$ git status`
 2. Add Changes
`$ git add filename.py` # Add specific file
`$ git add .` # Add all changes
`$ git add src/` # Add directory
 3. Commit Changes
`$ git commit -m "Descriptive message"`
 4. Push to Remote
`$ git push origin main`
- Useful Commands
 - View differences
`$ git diff` # Working directory vs staged
`$ git diff --staged` # Staged vs last commit
 - View history
`$ git log` # Detailed history
`$ git log --oneline` # Compact history
`$ git log --graph` # Visual representation
 - Undo changes
`$ git checkout -- filename` # Discard working changes
`$ git reset HEAD filename` # Unstage file

.gitignore: Excluding Files

- What is .gitignore?
 - Tells Git which files to ignore
 - Prevents tracking of unwanted files
 - Should be committed to repository
- Managing .gitignore
 - Create .gitignore
 - Add to repository
 - \$ git add .gitignore
 - \$ git commit -m "Add .gitignore file"

GitHub Basic Operations

- Create Remote Repository
 1. Login to GitHub
 2. Click "New" repository
 3. Repository settings
 - Name: my-first-project / Description: My first Python project / Visibility: Public (for learning)
 4. Click “Create repository”
- Connect Local to Remote
 - Add remote origin
`$ git remote add origin https://github.com/username/my-first-project.git`
 - Verify remote
`$ git remote -v`
 - Push to GitHub
`$ git branch -M main (# Ensure main branch) / $ git push -u origin main (# First push)`
`/ $ git push (# Subsequent pushes)`
- Clone Repository
`$ git clone https://github.com/username/repository.git`
- Sync Changes
 - Download changes from remote
`$ git pull origin main`
 - Push local changes
`$ git push origin main`